

Лабораторная работа 5

Построение графиков

Оразгелдиев Язгелди

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
4	Выполнение лабораторной работы	8
4.1	Задания для самостоятельного выполнения	26
5	Выводы	37

Список иллюстраций

4.1	Основные пакеты для работы с графиками в Julia	8
4.2	Основные пакеты для работы с графиками в Julia	9
4.3	Опции при построении графика	10
4.4	Опции при построении графика	11
4.5	Точечный график	12
4.6	Точечный график	13
4.7	Аппроксимация данных	14
4.8	Полярные координаты	15
4.9	График поверхности	16
4.10	Линии уровня	17
4.11	Векторные поля	18
4.12	Анимация	19
4.13	Гипоциклоида	20
4.14	Errorbars	21
4.15	Errorbars	22
4.16	Использование пакета Distributions	23
4.17	Подграфики	24
4.18	Подграфики	25
4.19	Подграфики	25
4.20	Задание №1 и №2	26
4.21	Задание №2	27
4.22	Задание №3	28
4.23	Задание №4	29
4.24	Задание №5	30
4.25	Задание №6	31
4.26	Задание №7	32
4.27	Задание №8	33
4.28	Задание №9	34
4.29	Задание №10	35
4.30	Задание №11	36

Список таблиц

1 Цель работы

Основная цель работы – освоить синтаксис языка Julia для построения графиков.

2 Задание

1. Используя JupyterLab, повторите примеры. При этом дополните графики обозначениями осей координат, легендой с названиями траекторий, названиями графиков и т.п.
2. Выполните задания для самостоятельной работы.

3 Теоретическое введение

Julia – высокоуровневый свободный язык программирования с динамической типизацией, созданный для математических вычислений [[julialang](#)]. Эффективен также и для написания программ общего назначения. Синтаксис языка схож с синтаксисом других математических языков, однако имеет некоторые существенные отличия.

Для выполнения заданий была использована официальная документация Julia [[juliadoc](#)].

4 Выполнение лабораторной работы

Выполним примеры из лабораторной работы для знакомства с пакетами по отрисовки графиков и их функциями (рис. [fig:001]-[fig:019]).



Рисунок 4.1: Основные пакеты для работы с графиками в Julia

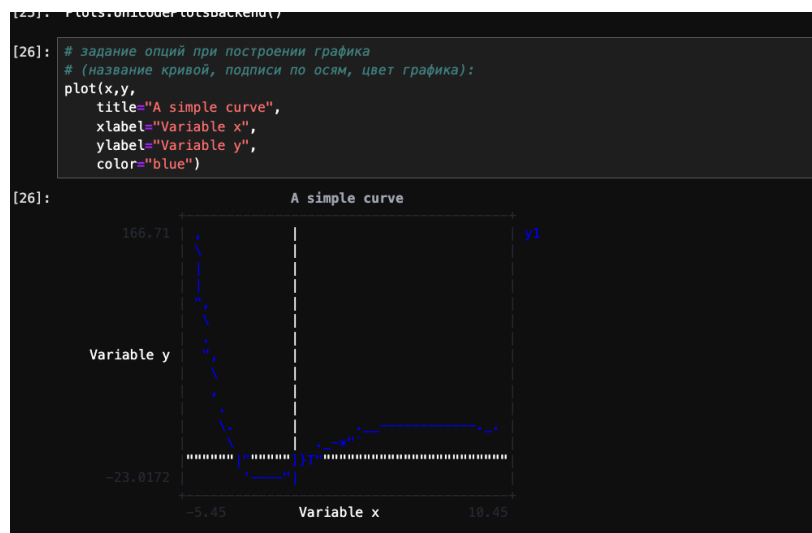


Рисунок 4.2: Основные пакеты для работы с графиками в Julia

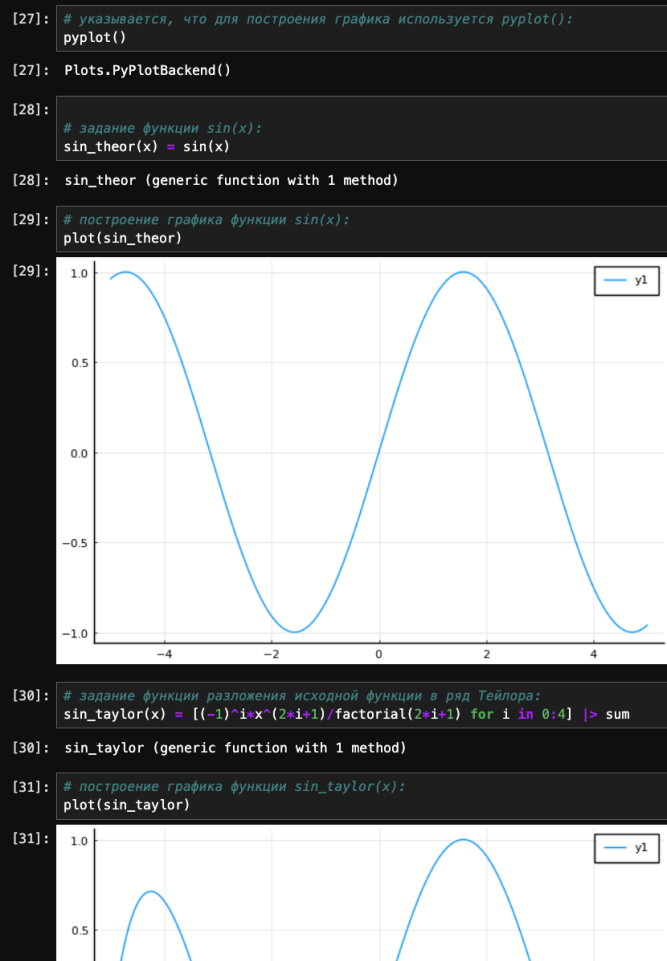


Рисунок 4.3: Опции при построении графика

```
[33]: plot(
# функция sin(x):
sin_taylor,
# подпись в легенде, цвет и тип линии:
label = "sin(x), разложение в ряд Тейлора",
line=(blue, 0.3, 10, :solid),
# размер графика:
size=(800, 500),
# параметры отображения значений по осям
xticks = (-5:0.5:5),
yticks = (-1:0.1:1),
xtickfont = font(12, "Times New Roman"),
ytickfont = font(12, "Times New Roman"),
# подписи по осям:
ylabel = "y",
xlabel = "x",
# название графика:
title = "Разложение в ряд Тейлора",
# поворот значений, заданный по оси x:
xrotation = rad2deg(pi/4),
# заливка области графика цветом:
fillrange = 0,
fillalpha = 0.3,
fillcolor = :green,
# задание цвета фона:
background_color = :ivory
)
plot!(
# функция sin_theor:
sin_theor,
# подпись в легенде, цвет и тип линии:
label = "sin(x), теоретическое значение",
line=(black, 1.0, 2, :dash)
)
)
```

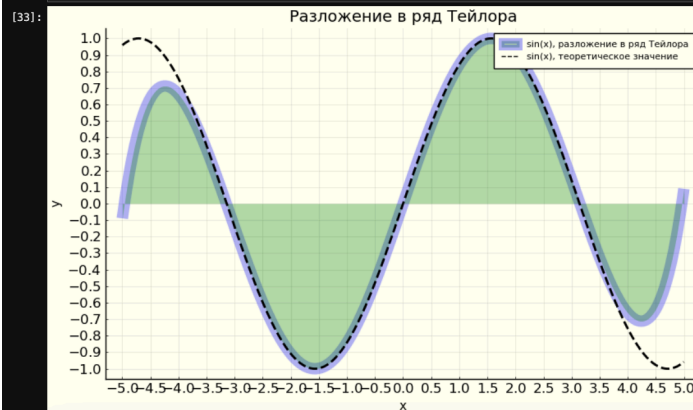


Рисунок 4.4: Опции при построении графика

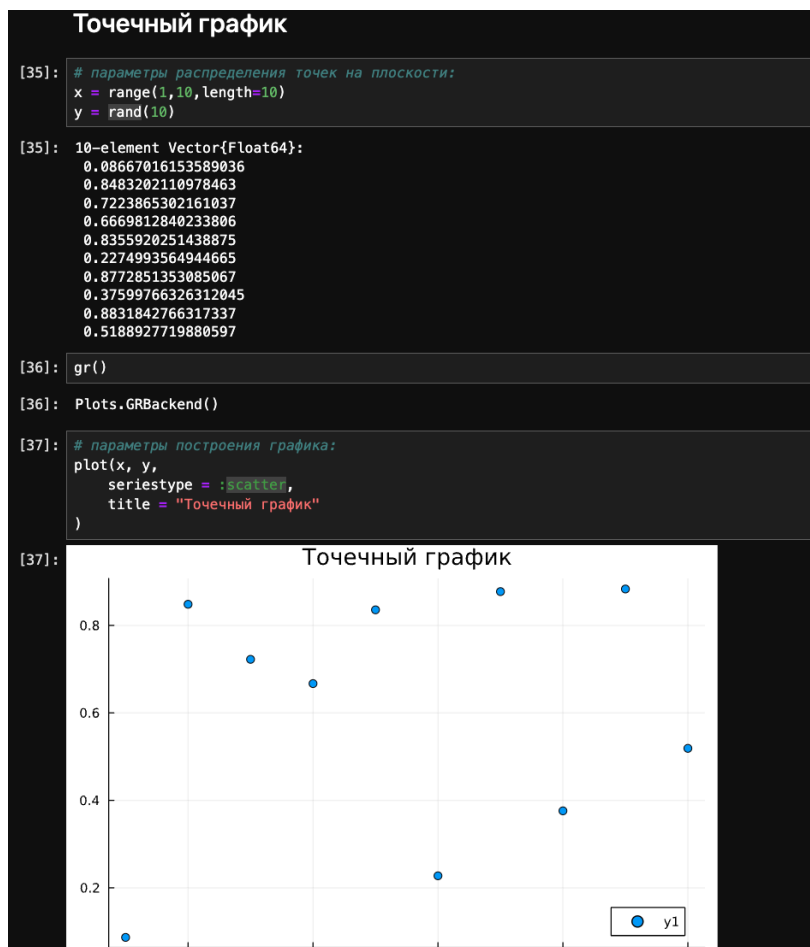


Рисунок 4.5: Точечный график

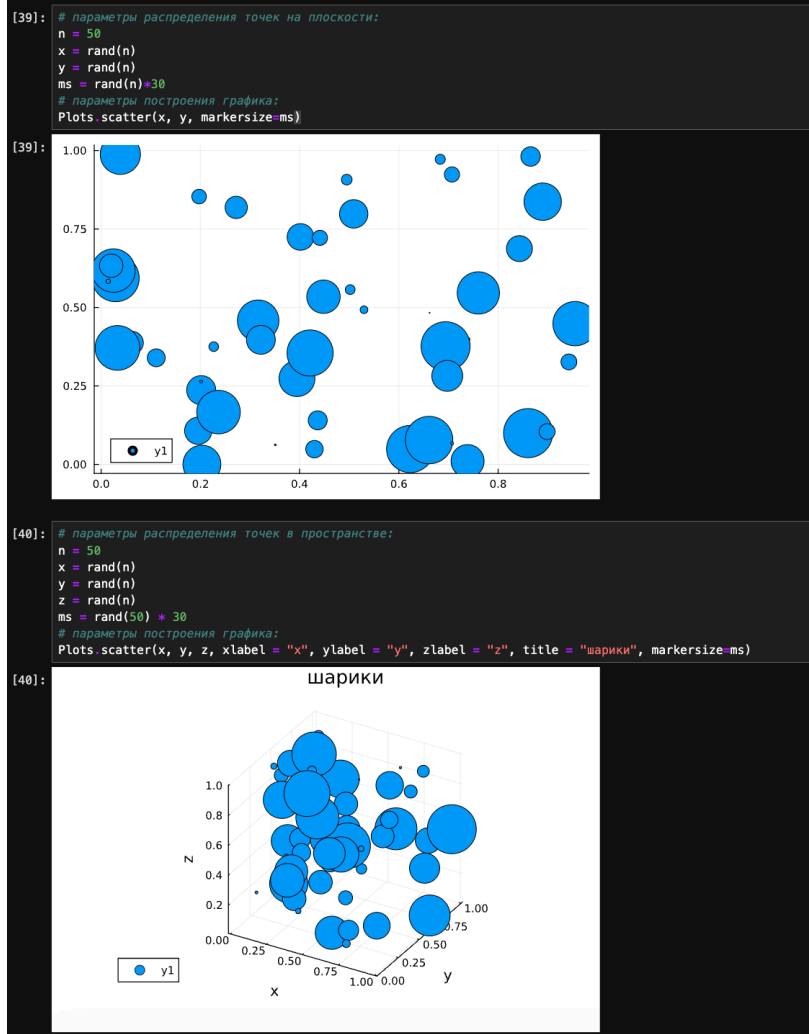
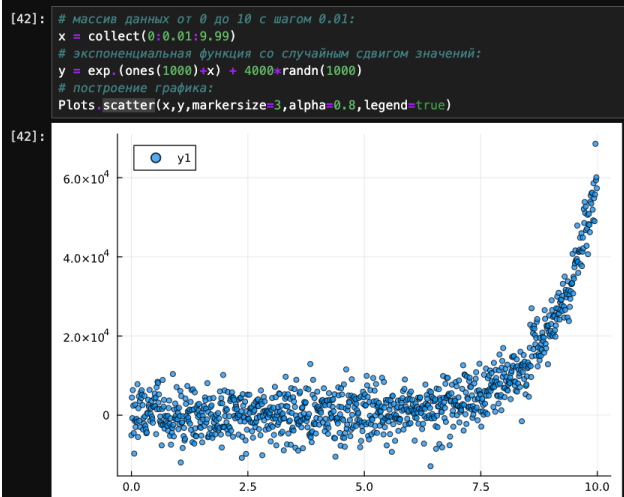
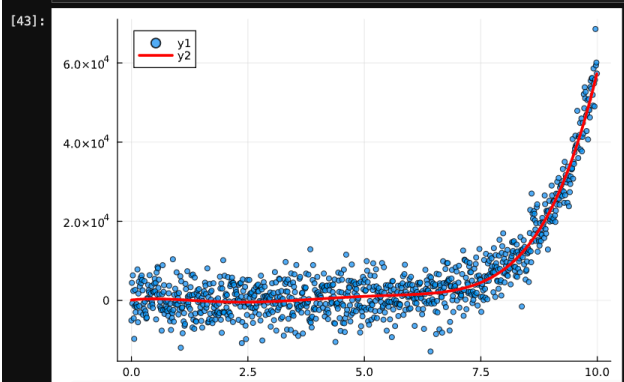


Рисунок 4.6: Точечный график



```
[43]: # определение массива для нахождения коэффициентов полинома:
A = [ones(1000) x x.^2 x.^3 x.^4 x.^5]
# решение матричного уравнения:
c = A \ y
# построение полинома:
g = c[1]*ones(1000) + c[2]*x + c[3]*x.^2 + c[4]*x.^3 + c[5]*x.^4 + c[6]*x.^5
# построение графика аппроксимирующей функции:
plot!(x,g,linewidth=3, color=:red)
```



```
[44]: # пример случайной траектории
```

Рисунок 4.7: Аппроксимация данных

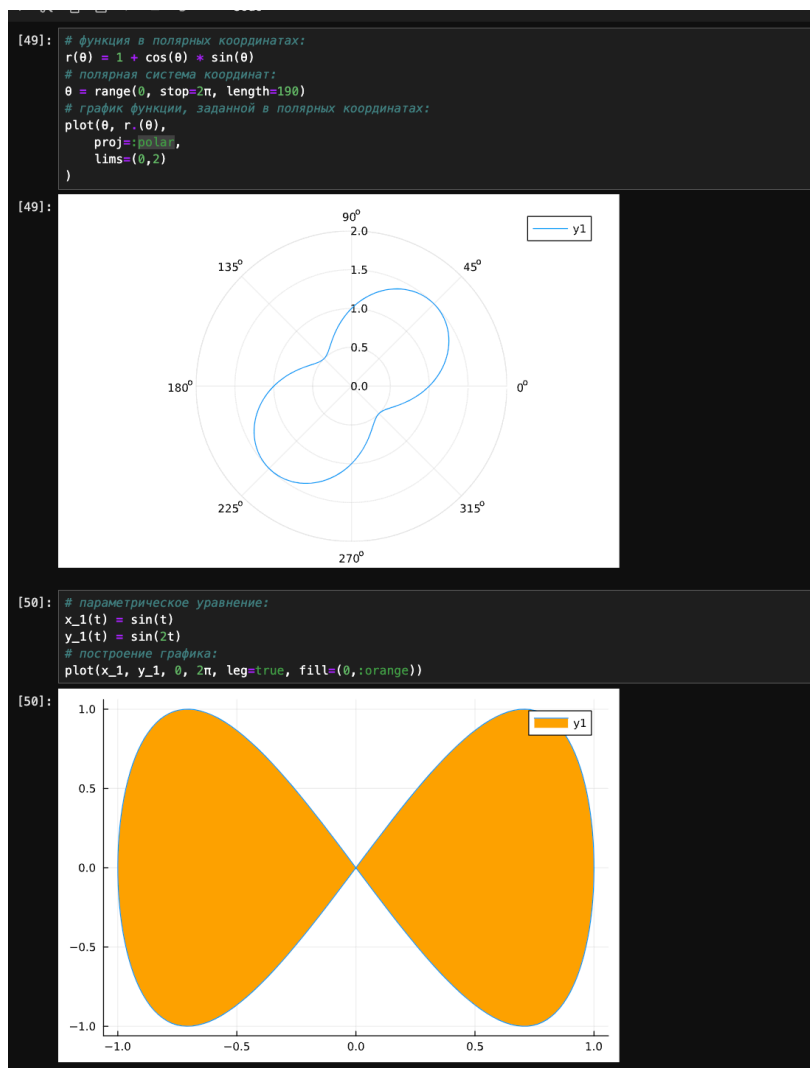


Рисунок 4.8: Полярные координаты

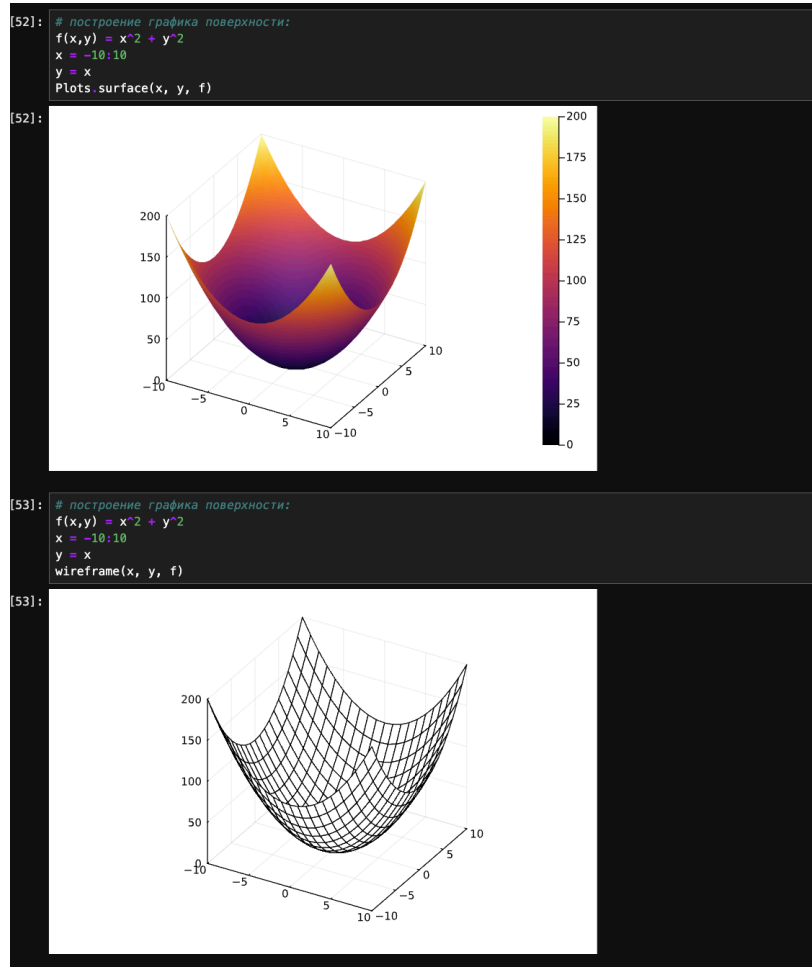


Рисунок 4.9: График поверхности

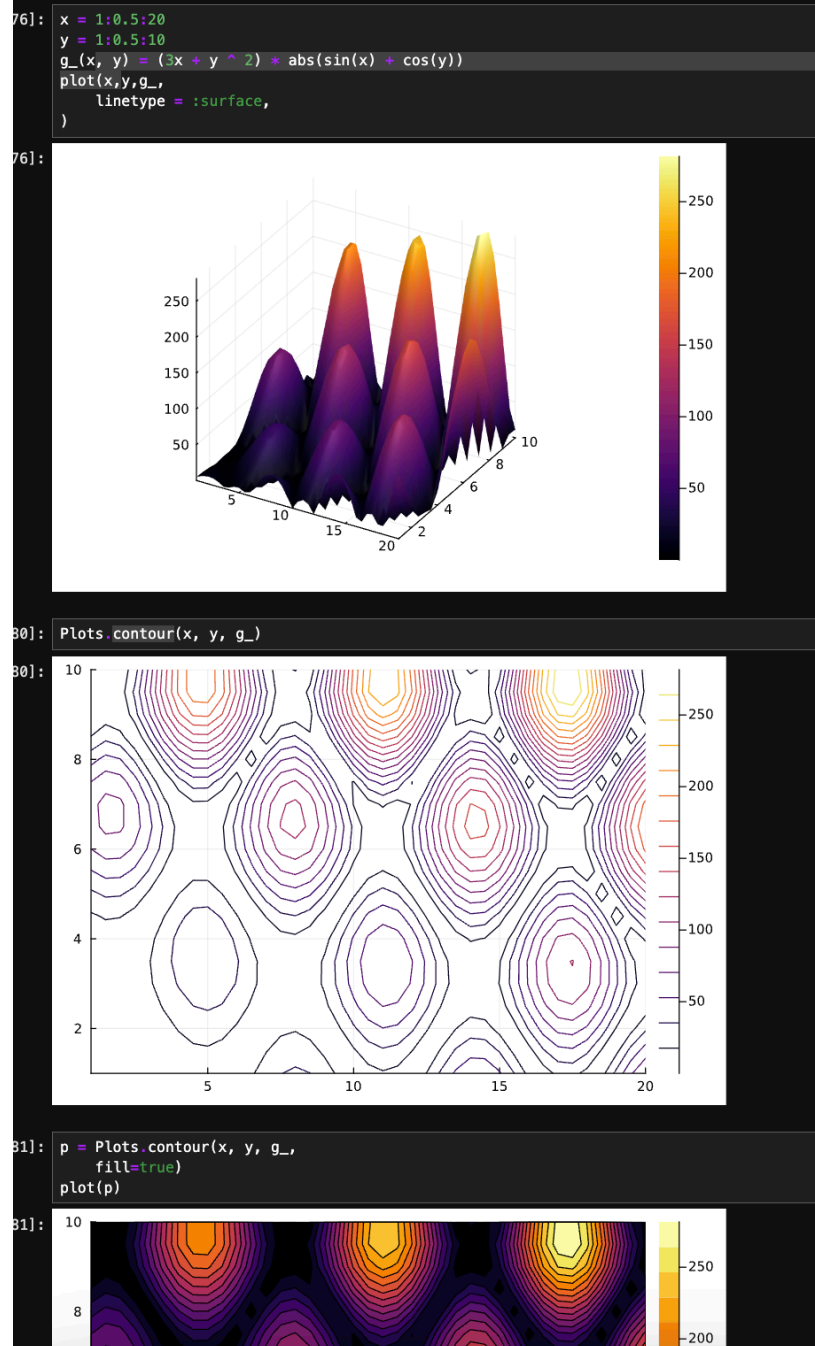


Рисунок 4.10: Линии уровня

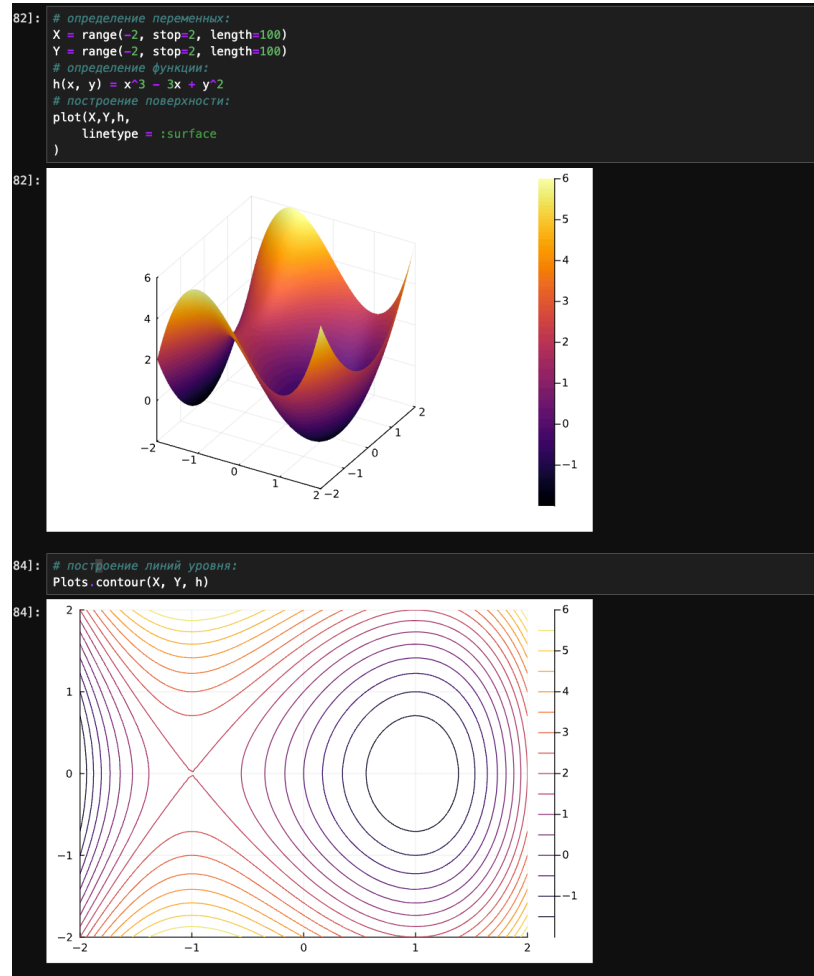


Рисунок 4.11: Векторные поля

Анимация

```
01: pyplot()
# построение поверхности:
i = 0
X = Y = range(-5, stop=5, length=40)
Plots.surface(X, Y, (x,y) -> sin(x+10sin(i))+cos(y))
# анимация:
X = Y = range(-5, stop=5, length=40)
@gif for i in range(0, stop=2π, length=100)
    Plots.surface(X, Y, (x,y) -> sin(x+10sin(i))+cos(y))
end
[ Info: Saved animation to /Users/darabeliceva/work/study/julia/tmp.gif
```

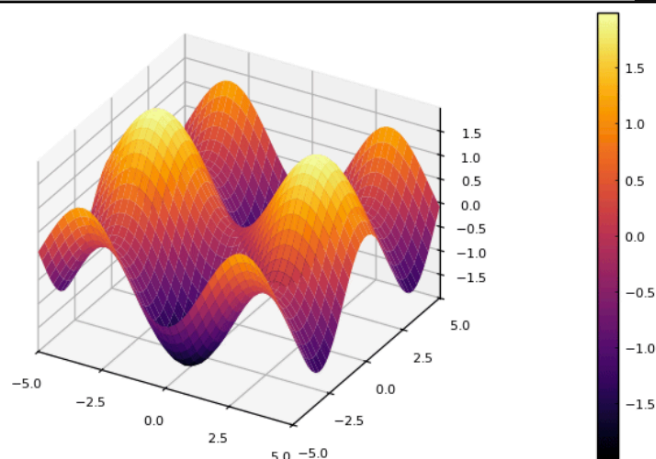
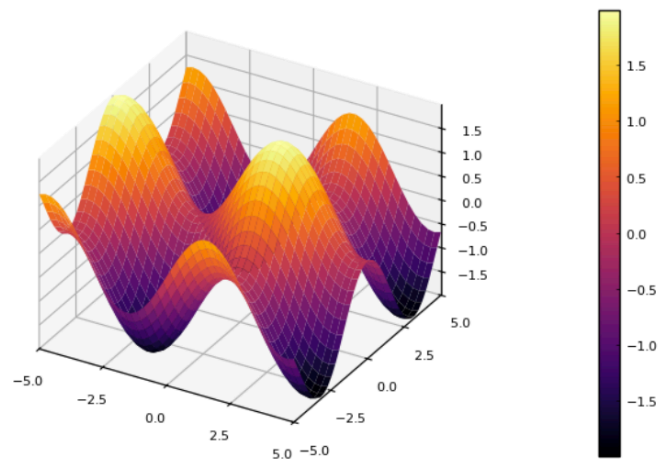


Рисунок 4.12: Анимация

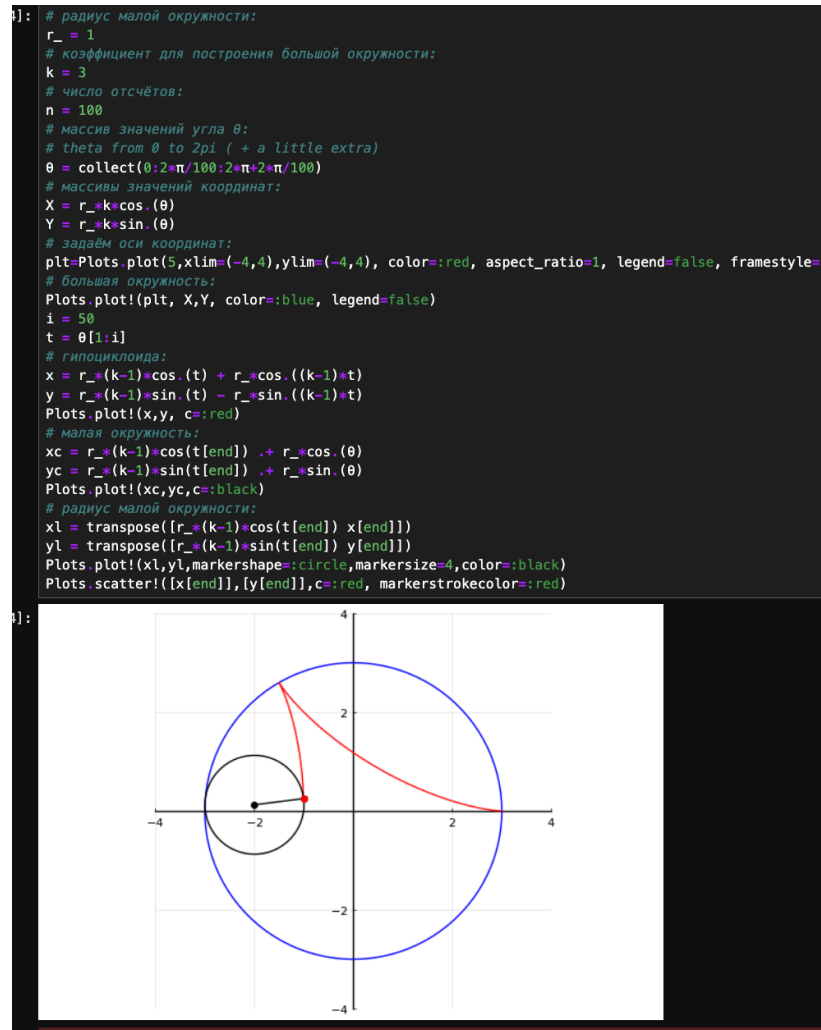


Рисунок 4.13: Гипоциклоида

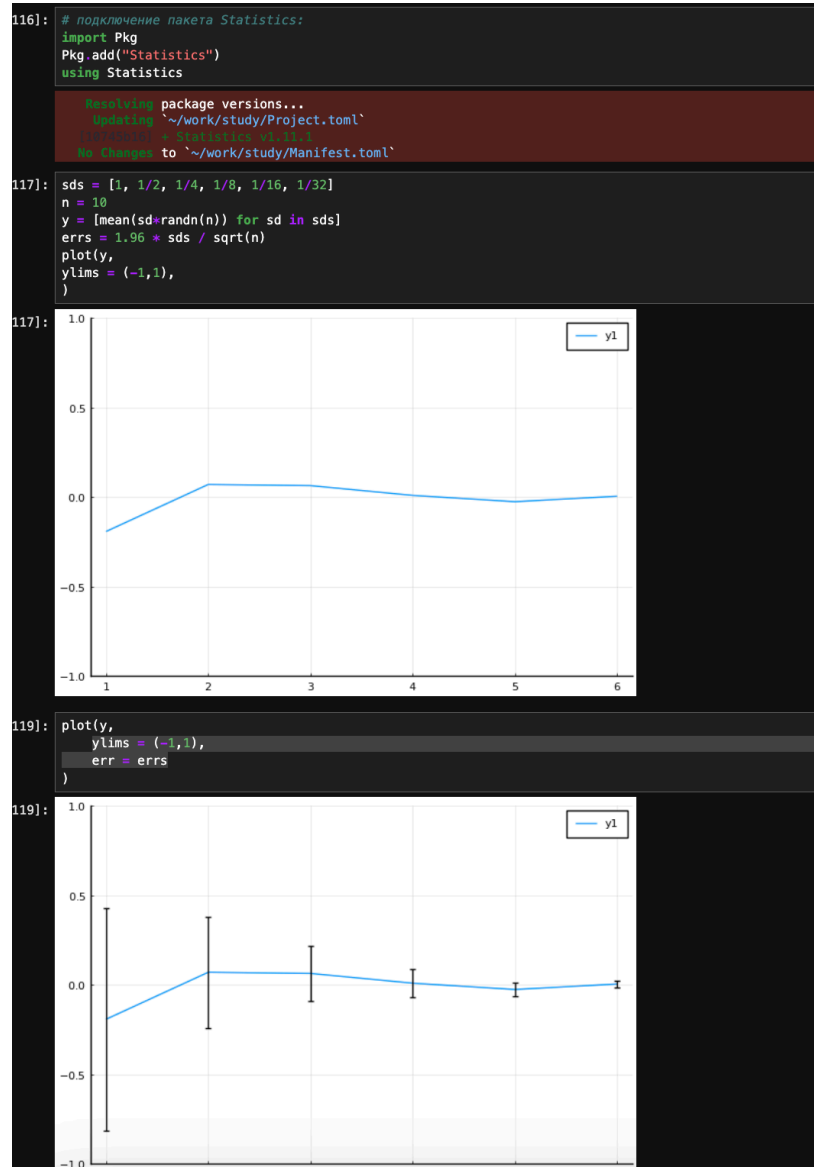


Рисунок 4.14: Errorbars

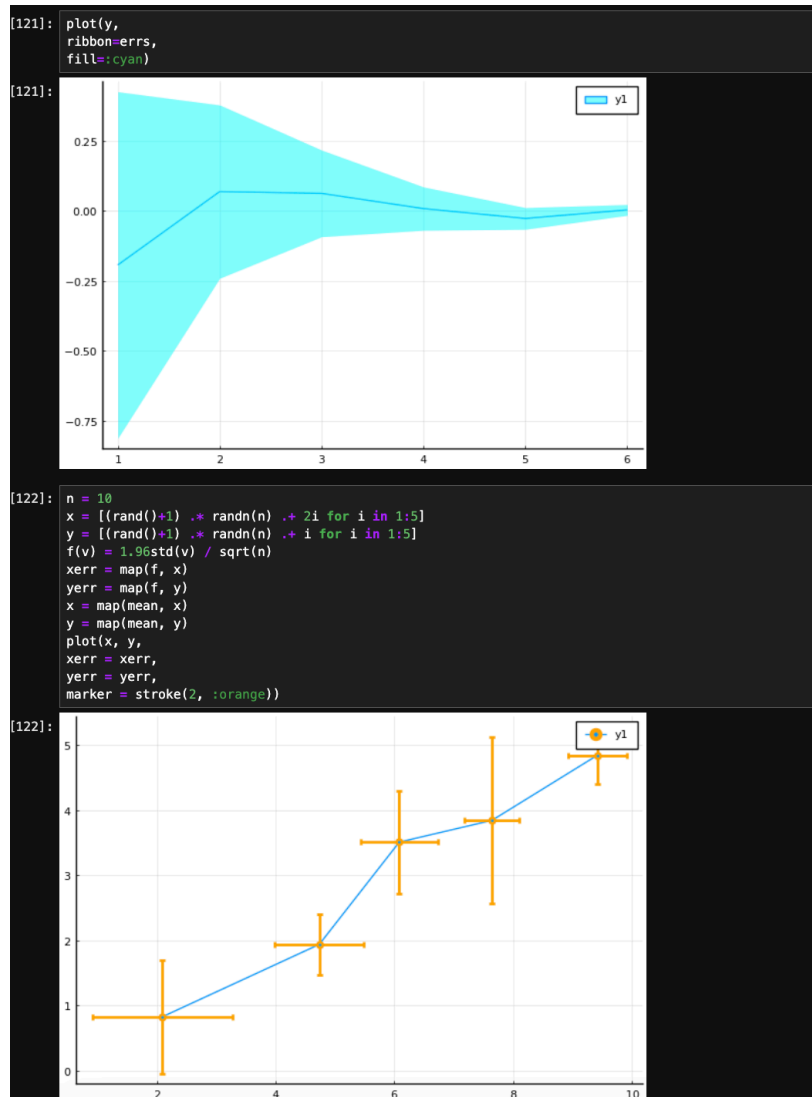


Рисунок 4.15: Errorbars

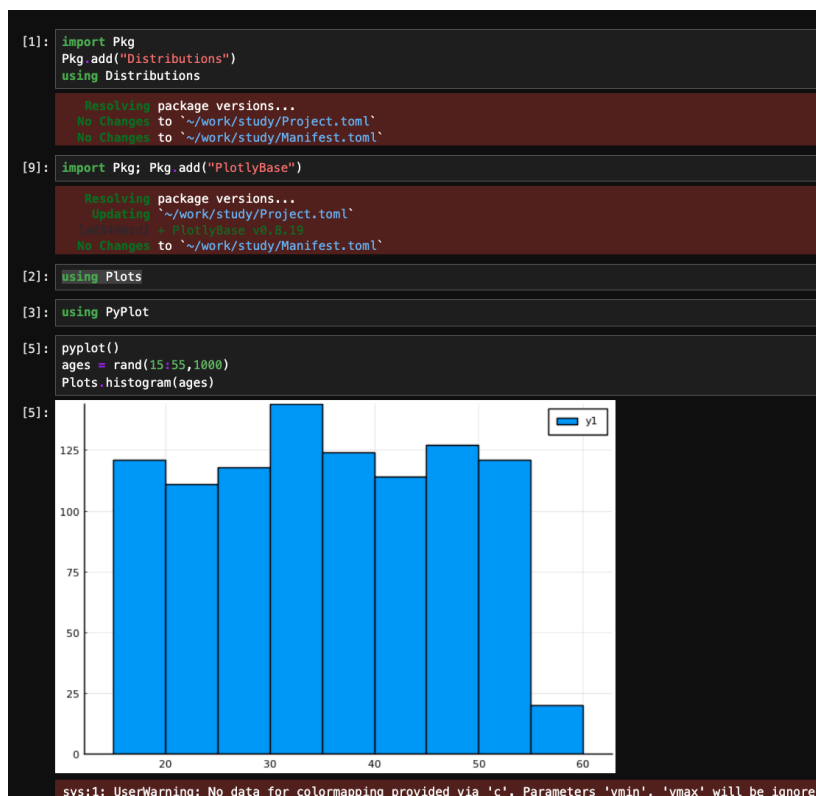


Рисунок 4.16: Использование пакета Distributions

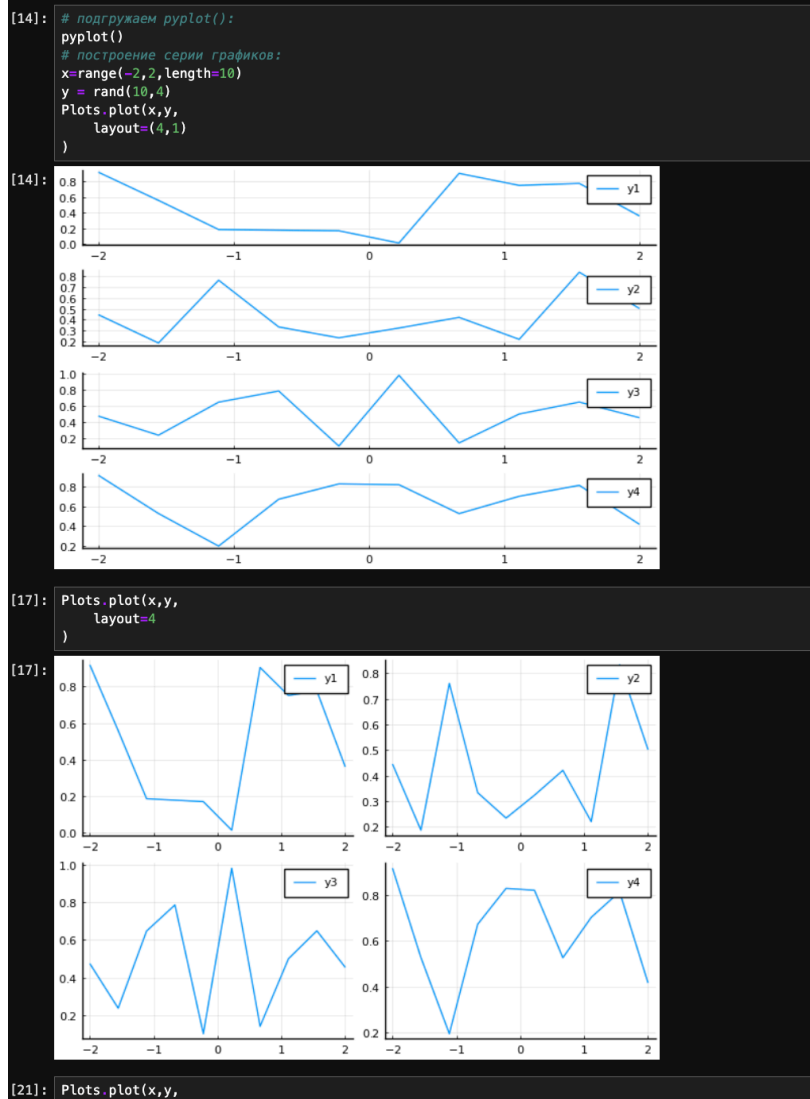


Рисунок 4.17: Подграфики

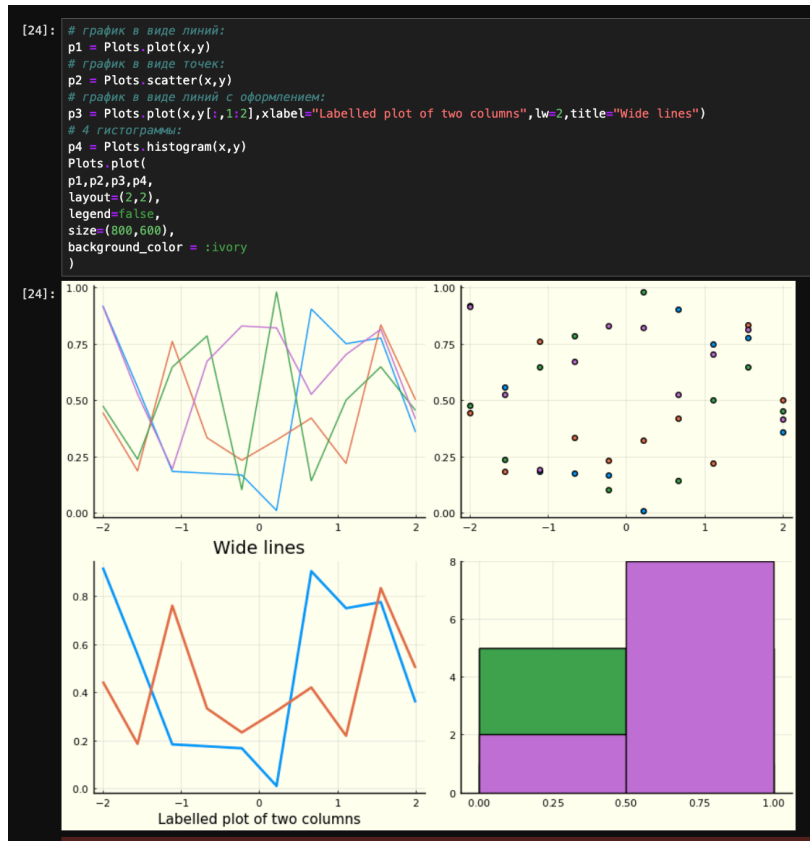


Рисунок 4.18: Подграфики



Рисунок 4.19: Подграфики

4.1 Задания для самостоятельного выполнения

Выполним задания.

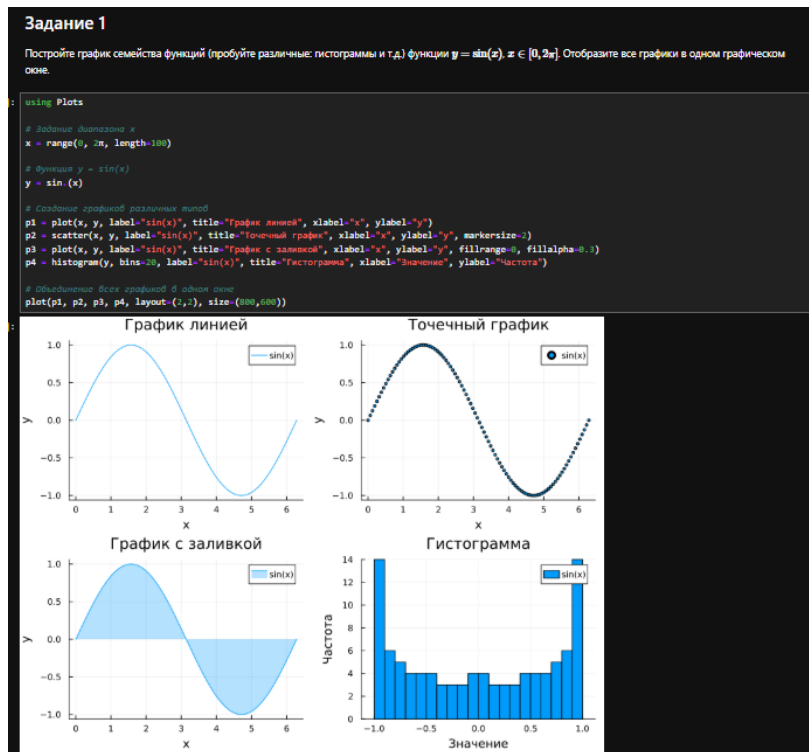


Рисунок 4.20: Задание №1 и №2

Задание 2 ¶

Постройте графики функции $y = \sin(x)$, $x \in [0, 2\pi]$ со всеми возможными (сколько сможете придумать) типами линий графиков. Отобразите все графики в одном графическом окне.

```
using Plots
x = range(0, 2π, length=100)
y = sin.(x)

# Различные типы линий
p1 = plot(x, y, linestyle=:solid, label="solid", title="Solid", lw=2)
p2 = plot(x, y, linestyle=:dash, label="dash", title="Dash", lw=2)
p3 = plot(x, y, linestyle=:dot, label="dot", title="Dot", lw=2)
p4 = plot(x, y, linestyle=:dashdot, label="dashdot", title="Dashdot", lw=2)
p5 = plot(x, y, linestyle=:dashdotdot, label="dashdotdot", title="Dashdotdot", lw=2)

# Объединение графиков
plot(p1, p2, p3, p4, p5, layout=(3,2), size=(900,700), legend=:true)
```

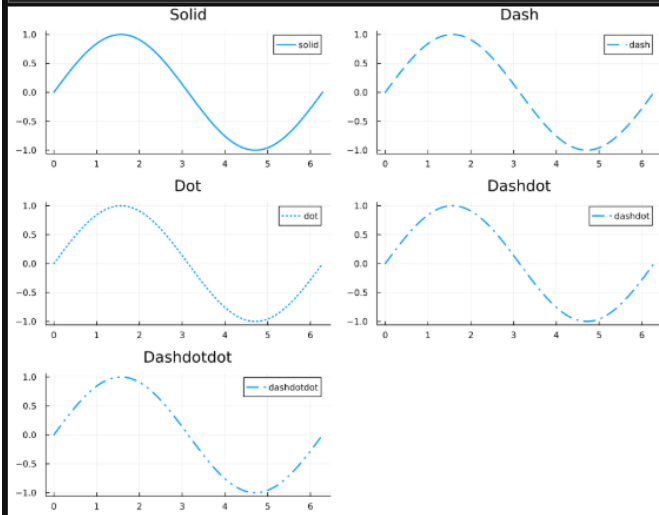


Рисунок 4.21: Задание №2

Задание 3

Постройте график функции $f(x) = \pi x^2 \ln(x)$, назовите оси соответственно. Пусть цвет рамки будет зелёный, а цвет самого графика — красным. Задайте расстояние для надписи и оси так, чтобы надписи полностью умещались в графическом окне. Задайте шрифт надписей. Задайте частоту отсчетов на осях координат.

```
using Plots

# Диапазон x (избегаем x=0, так как ln(0) не определен)
x = range(0.1, 5, length=100)

# Функция f(x) = π*x^2*ln(x)
f(x) = π * x^2 * log(x)
y = f(x)

# Построение графика
plot(x, y,
     xlabel="x",
     ylabel="f(x) = π*x^2*ln(x)",
     title="График функции f(x) = π*x^2*ln(x)",
     color=:red,
     lw=2,
     framestyle=:box,
     fg_color=:white,
     border=:green,
     xticks=:auto,
     yticks=:auto,
     tickfont=:font(10, "Calibri"),
     guidefont=:font(11, "Calibri"),
     titlefont=:font(14, "Calibri"),
     legend=:none,
     size=(700,500))
```

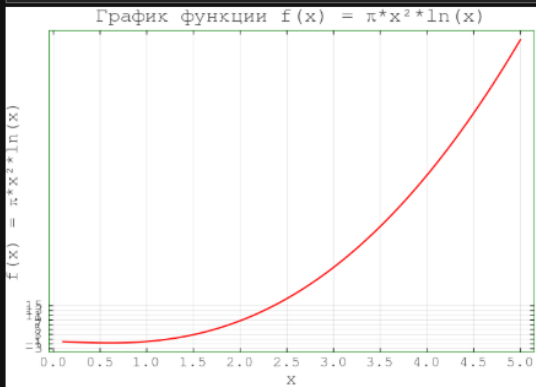


Рисунок 4.22: Задание №3

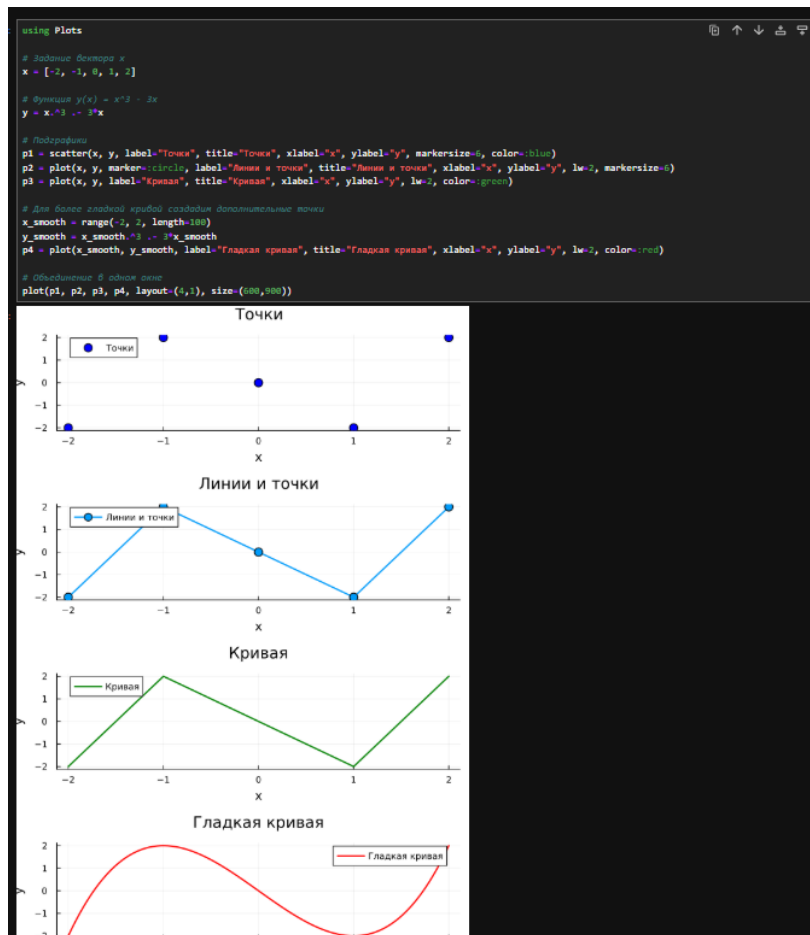


Рисунок 4.23: Задание №4

Задание 5

Задать вектор $x = (3, 3.1, 3.2, \dots, 6)$. Постройте графики функций $y_1(x) = \pi x$ и $y_2(x) = \cos(x)$ в указанном диапазоне координат одновременно разными образамис.

- постройте оба графика разного цвета на одном рисунке, добавьте легенду и сетку для каждого графика;
- укажите недостаток у данного построения;
- постройте оба графика с двумя осями ординат.

```
using Plots

# Задание вектора x
x = 3:0.1:6

# Функции
y1 = π * x
y2 = cos(x)

# График 1: оба графика на одной оси Y
p1 = plot(x, y1, label="y = πx", xlabel="x", ylabel="y", lw=3, color=:blue, legend=:topleft, grid=true)
plot!(p1, x, y2, label="y = cos(x)", lw=3, color=:red)
title(p1, "Два графика на одной оси Y")

# Недостаток: масштабы функций сильно различаются, cos(x) ∈ [-1, 1], а πx ∈ [9.4, 18.8]
# График 2: оба графика с двумя осями ординат
p2 = plot(x, y1, label="y = πx", xlabel="x", ylabel="y1 = πx", lw=3, color=:blue, legend=:topleft)
plot!(twink(), x, y2, label="y = cos(x)", ylabel="y2 = cos(x)", lw=3, color=:red, legend=:topright, grid=false)
title(p2, "Два графика с двумя осями Y")

# Объединение
plot(p1, p2, layout=(2,1), size=(800,700))
```

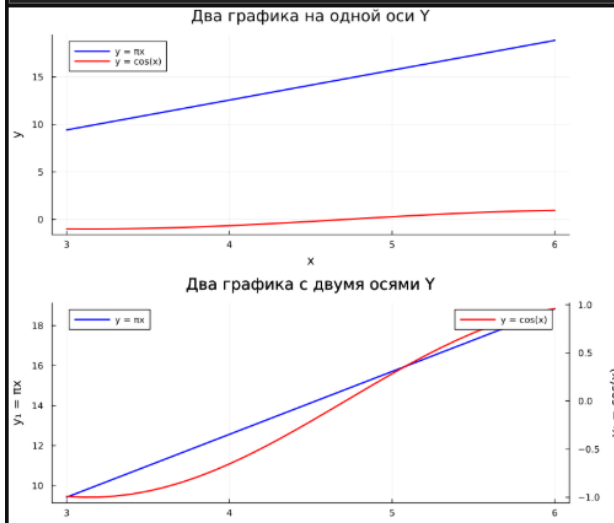


Рисунок 4.24: Задание №5

Задание 6

Постройте график некоторых экспериментальных данных (придумайте сами), учитывая ошибки измерения.

```
using Plots
using Statistics

# Экспериментальные данные (например, зависимость температуры от времени)
time = 0:10:100 # время в минутах
temperature = [20.5, 22.3, 24.1, 26.8, 29.2, 31.5, 33.9, 36.2, 38.1, 39.8, 41.2] # температура в °C

# Ошибки измерения (стандартное отклонение)
errors = [0.5, 0.6, 0.4, 0.7, 0.5, 0.6, 0.8, 0.7, 0.6, 0.9, 0.8]

# График с ошибками
plot(time, temperature,
     yerr=errors,
     marker=:circle,
     markersize=6,
     label="Экспериментальные данные",
     xlabel="Время (мин)",
     ylabel="Температура (°C)",
     title="Зависимость температуры от времени",
     lw=2,
     legend=:bottomright,
     grid=true,
     size=(700,500))
```

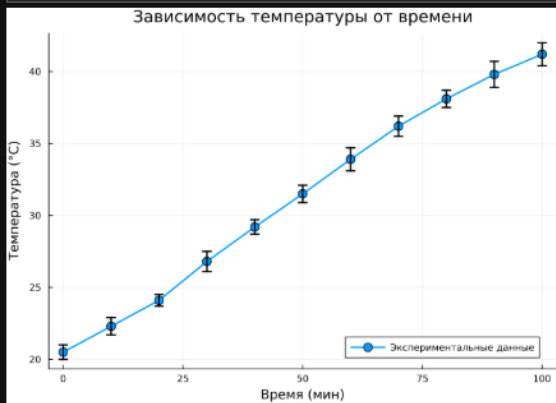


Рисунок 4.25: Задание №6

Задание 7

Постройте точечный график случайных данных. Подпишите оси, легенду, название графика.

```
using Plots
# Генерация случайных данных
n = 50
x = rand(n) * 10 # случайные значения x от 0 до 10
y = rand(n) * 20 # случайные значения y от 0 до 20

# Точечный график
scatter(x, y,
        label="Случайные данные",
        xlabel="Переменная X",
        ylabel="Переменная Y",
        title="Точечный график случайных данных",
        markerize=:blue,
        markerstrokewidth=:black,
        legend=:topright,
        grid=:true,
        size=(700,500)
)
```

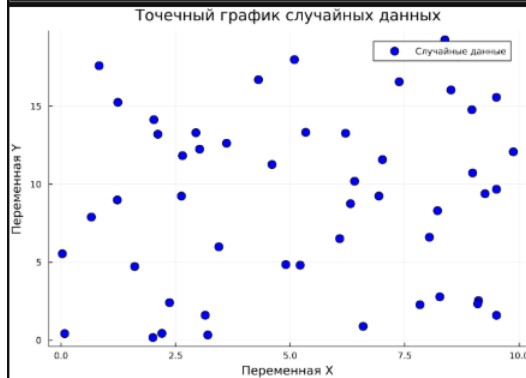


Рисунок 4.26: Задание №7

Задание 8

Постройте 3-мерный точечный график случайных данных. Подпишите оси, легенду, название графика.

```
using Plots

# Генерация случайных данных в 3D
n = 100
x = rand(n) .* 10
y = rand(n) .* 10
z = rand(n) .* 10

# Размер маркеров (опционально)
ms = rand(n) .* 5 .+ 2

# 3D точечный график
scatter(x, y, z,
        label="3D случайные данные",
        xlabel="X",
        ylabel="Y",
        zlabel="Z",
        title="3D точечный график случайных данных",
        markersize=ms,
        markercolor=:viridis,
        markerstrokewidth=0.5,
        legend=:topright,
        size=(700,600))
)
```

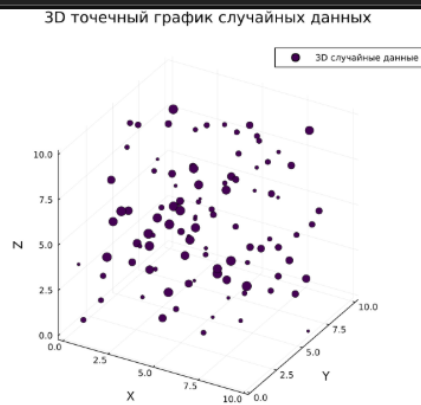


Рисунок 4.27: Задание №8

Задание 9

Создайте анимацию с построением синусоиды. То есть вы строите последовательность графиков синусоиды, начиная с малого модуля k и заканчивая значением модуля k . После соедините их в анимацию и сохраните модуль k .

```
using Plots

# Параметры
x = range(0, 4π, length=200)
k_values = range(0.1, 3, length=50) # модуль k от 0.1 до 3

# Создание анимации
@gif for k in k_values
    y = k .* sin(x)
    plot(x, y,
        xlabel="x",
        ylabel="y",
        title="y = $(round(k, digits=2)) * sin(x)",
        lw=2,
        color=:blue,
        ylim=(-3.5, 3.5),
        xlim=(0, 4π),
        legend=false,
        grid=true,
        size=(700, 400)
    )
end
```

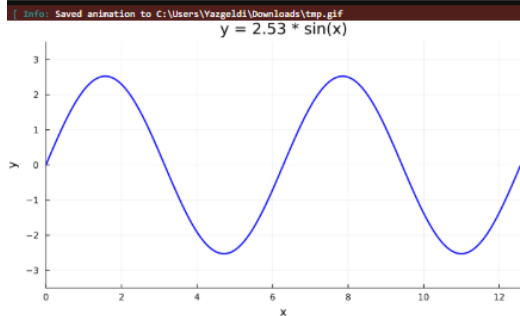


Рисунок 4.28: Задание №9

Задание 10

Постройте анимированную гипоциклоиду для 2 целых значений модуля k и 2 рациональных значений модуля k .

```
using Plots

# Функция для построения гипоциклоиды
function hypocycloid_animation(k, r=1, filename="hypocycloid_k$(k).gif")
    n = 100
    θ = collect(0:2π/n:2π/n)

    # Большая окружность
    X = r*k*cos(θ)
    Y = r*k*sin(θ)

    anim = @animate for i in 1:n
        t = θ[i:i]

        # Координаты гипоциклоиды
        x = r*(k-1)*cos.(t) .+ r*cos.((k-1)*t)
        y = r*(k-1)*sin.(t) .- r*sin.((k-1)*t)

        # Мелкая окружность
        xc = r*(k-1)*cos.(t[end]) .+ r*cos(θ)
        yc = r*(k-1)*sin.(t[end]) .+ r*sin(θ)

        plt = plot(xlim=(-k*r-1, k*r+1), ylim=(-k*r-1, k*r+1),
                  aspect_ratio=:equal, legend=:false, framestyle=:origin)
        plot!(plt, X, Y, c=:black, lw=2) # Большая окружность
        plot!(plt, x, y, c=:red, lw=2) # Гипоциклоида
        plot!(plt, xc, yc, c=:black, lw=1) # Мелкая окружность

        # Подпись и метки
        x1 = [r*(k-1)*cos.(t[end]), x[end]]
        y1 = [r*(k-1)*sin.(t[end]), y[end]]
        plot!(plt, x1, y1, c=:black, lw=1, marker=:circle, markersize=4)
        scatter!(plt, [x[end]], [y[end]], c=:red, markersize=6, markerstrokecolor=:red)

        title!("Гипоциклоида k = $(k)")
    end

    gif(anim, filename, fps=15)
    println("Анимация сохранена: $filename")
end

# Построение для 2 целых значений k
hypocycloid_animation()
hypocycloid_animation()

# Построение для 2 рациональных значений k
hypocycloid_animation(1.5)
hypocycloid_animation(1.3)
```

Анимация сохранена: hypocycloid_k2.gif
[Info: Saved animation to C:\Users\Varzeldi\Downloads\hypocycloid_k2.gif

Анимация сохранена: hypocycloid_k3.gif
[Info: Saved animation to C:\Users\Varzeldi\Downloads\hypocycloid_k3.gif

Анимация сохранена: hypocycloid_k2.5.gif
[Info: Saved animation to C:\Users\Varzeldi\Downloads\hypocycloid_k2.5.gif

Анимация сохранена: hypocycloid_k3.3.gif
[Info: Saved animation to C:\Users\Varzeldi\Downloads\hypocycloid_k3.3.gif

Рисунок 4.29: Задание №10

```

using Plots

# Функция для построения эпициклоиды
function epicycloid_animation(k, r=1, filename="epicycloid_k2.gif")
    n = 100
    θ = collect(0:2π/n:2π-2π/n)

    # Большая окружность
    X = r*k*cos.(θ)
    Y = r*k*sin.(θ)

    anim = @animate for i in 1:n
        t = θ[i:i]

        # Координаты эпициклоиды
        x = r*(k+1)*cos.(t) .- r*cos.((k+1)*t)
        y = r*(k+1)*sin.(t) .- r*sin.((k+1)*t)

        # Малая окружность
        xc = r*(k+1)*cos(t[end]) .+ r*cos.(θ)
        yc = r*(k+1)*sin(t[end]) .+ r*sin.(θ)

        plt = plot(xlim=(-(k+2)*r-1, (k+2)*r+1), ylim=(-(k+2)*r-1, (k+2)*r+1),
            aspect_ratio=:equal, legend=false, framestyle=:origin)
        plot!(plt, X, Y, c=:blue, lw=2) # Большая окружность
        plot!(plt, x, y, c=:red, lw=2) # Эпициклоида
        plot!(plt, xc, yc, c=:black, lw=1) # Малая окружность

        # Радиус и точка
        x1 = [r*(k+1)*cos(t[end]), x[end]]
        y1 = [r*(k+1)*sin(t[end]), y[end]]
        plot!(plt, x1, y1, c=:black, lw=1, marker=:circle, markersize=4)
        scatter!(plt, [x[end]], [y[end]], c=:red, markersize=6, markerstrokecolor=:red)

        title!("Эпициклоида k = $k")
    end

    gif(anim, filename, fps=15)
    println("Анимация сохранена: $filename")
end

# Построение для 2 целых значений k
epicycloid_animation(2)
epicycloid_animation(3)

# Построение для 2 рациональных значений k
epicycloid_animation(1.5)
epicycloid_animation(1.3)

Анимация сохранена: epicycloid_k2.gif
[ Info: Saved animation to c:\Users\Yazgeldi\Downloads\epicycloid_k2.gif

```

Рисунок 4.30: Задание №11

5 Выводы

В результате выполнения данной лабораторной работы я освоил синтаксис языка Julia для построения графиков.